implement system call

> 해당 프로젝트는 어떠한 AI 도움 없이 작성했음을 다시 한번 밝힙니다.

- System Call Execution Flow (Only for this assignment) 부분은 0x1F, 0x1E, 0x1D에 정리했습니다!

0x00. requirements

0x01. ps user-level program

1. print process information w/o arg

- no arg -> all active processes

```
$ ps
pid,state,name
1,sleep,init
2,sleep,sh
3,run,ps
```

2. print process information w/ arg(s)

```
$ ps 1 2
pid,state,name
1,sleep,init
2,sleep,sh
```

```
$ ps 10
pid,state,name
```

0x02. ps system call

1. role

> iterate through the kernel's process table to find and print the status of the requested process(es) directly to the console

2. signature

```
int ps(int pid)
```

- arg `pid`

    - PID of the specific process to query
    - if 0, all active processes
- return value
    - 0 if target process(es) exist and are successfully printed
    - 1 if the specified process does not exist

0x10. DESIGN

0x11. conceptual design (overall)

1. system call occurs in user process execution

in system call interface

2. CPU operation

1. mode → kernel mode (dual-mode operation)
2. current context 저장
    - CPU가 자동으로 저장:
        - PC; Program Counter
        - flags
        - 일부 register (architecture 의존)
    - 나머지는 kernel entry code가 직접 저장
3. kernel stack으로 switch
    - CPU가 현재 process의 kernel stack pointer를 자동으로 로드 (kernel은 process마다 별도 kernel stack 사용하므로)
4. entry jump(trap)
    - trap 발생 시 미리 정해진 entry 주소로 jump
    - CPU는 interrupt vector table / trap table을 참조 (OS가 이 테이블을 세팅)

3. kernel code execution: syscall dispatch

1. syscall number 확인
2. syscall table lookup
3. handler 실행

4. return to user

1. return instruction (iret, sysret, sret)
    - dual-mode operation: mode → user mode
2. saved context 복원

0x12. idea

system call

- idea1: user space위 memory block에 kernel이 쓰게끔하자
  - -> 기각
  - (·.· 0x32, 0x33참고)
    - implement system call > 0x32. design1 — buffer pointer를 정의해서 register 로 넘겨서 kernel이 거기에 쓰게 하자.
    - implement system call > 0x33. 방법1의 오류
- idea2: kernel이 console에 직접 접근할 수 있으므로 직접 print해서 출력

0x1F. system call execution flow

1. user code(`ps.c`)
   - `ps(atoi(argv[i]))`
2. usys.S /pl
   - li a7, SYS_ps // syscall 번호
     - `#define SYS_ps 22` in syscall.h에 정의되어 있는 macro constant
   - ecall // kernel 진입
   - ret // 결과 반환
3. ecall
   - user mode에서 CPU trap 발생
   - CPU가 stvec에 들어 있는 trap entry address로 control transfer
   - 그 주소는 trampoline의 uservec
4. uservec (trampoline.S)
   - user register를 trapframe에 저장
   - kernel stack으로 전환
   - kernel page table로 전환
   - usertrap() 호출
5. usertrap()
   - syscall trap인지 확인
   - sepc를 저장하고 다음 instruction으로 넘기기 위해 epc += 4
   - syscall() 호출
6. syscall()
   - trapframe->a7에서 syscall number를 읽음
   - syscalls[num] dispatch
   - return 값을 trapframe->a0에 저장
7. sys_ps 호출
   1. arg는 이미 trampoline.S에서 trapframe으로 옮겨짐: `argint(0, &pid)`
   2. 이 trapframe에 저장되어 있던 argument (a0)를 가져옴.

8. ps() in `proc.c` 호출

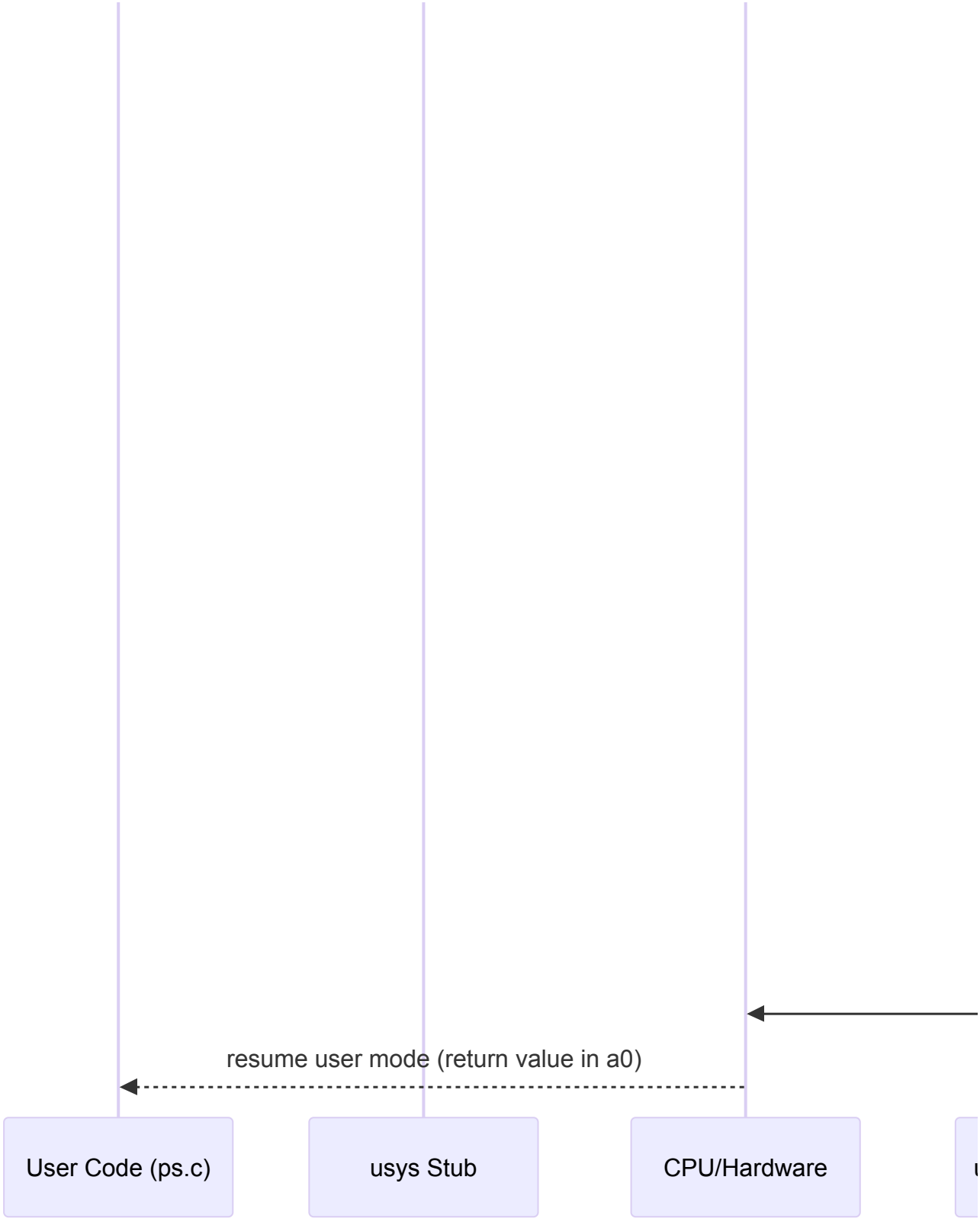
0x1E. system call termination flow

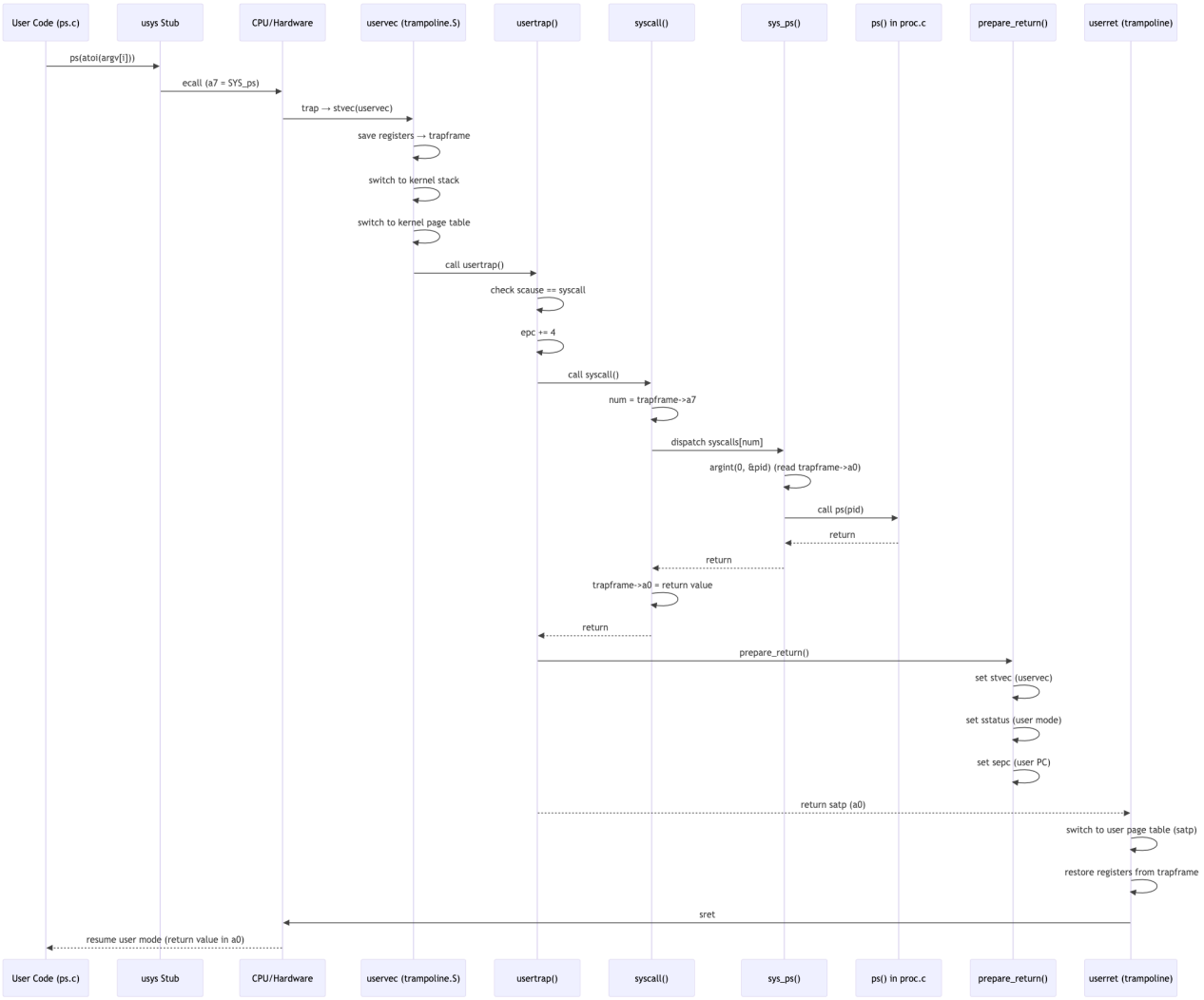
1. ps() returns to sys_ps()
2. sys_ps() returns to syscall()
    1. syscall() stores return value in p->trapframe->a0
3. syscall() returns to usertrap()
4. usertrap() calls prepare_return()
    - stvec를 다음 user trap용 entry로 세팅
    - kernel-side trapframe metadata 저장
    - sstatus를 user mode 복귀 가능하게 세팅
    - sepc를 user PC로 세팅
5. usertrap() returns satp = MAKE_SATP(p->pagetable)
6. trampoline userret(satp)
    - user page table로 전환 (a0에 저장됨; C → assembly argument 전달은 register(a0) 사용)
    - trapframe에서 user register 복원
    - a0도 복원
    - sret 실행
7. user mode로 복귀, user code는 syscall return value를 받음

0x1D. visualization

> 잘렸다면 별도로 sequence_diagram_system_call_IMPL.png 파일을 참고해주세요!


User Code (ps.c)
usys Stub
CPU/Hardware
ps(atoi(argv[i]))
ecall (a7 = SYS_ps)
trap → stvec(use
sa
sw


resume user mode (return value in a0)
User Code (ps.c)
usys Stub
CPU/Hardware



0x20. IMPL

0x21. define system call

> in `kernel/proc.c`
> xv6의 아키텍처는 subsystem별로 코드가 정의되어 있음(resource + responsibility 기준 코드 분할). 따라서 proc의 상태를 표현하기 위한 system call이므로 해당 위치에 정의한다.

```
int
ps(int pid)
{
    static char *states[] = {
        [UNUSED] "unused",
        [USED] "used",
        [SLEEPING] "sleep",
        [RUNNABLE] "runble",
        [RUNNING] "run",
        [ZOMBIE] "zombie"
    };
```

```
    struct proc *p;

    int unfound = 1;

    if (pid == 0) {
        for(p = proc; p < &proc[NPROC]; p++) {
            int pid_local;
            char state_local[16];
            char name_local[16];
            if(p->state > UNUSED && p->state <= NELEM(states)) {
                acquire(&p->lock);
                unfound = 0;
                pid_local = p->pid;
                safestrcpy(state_local, states[p->state], 16);
                safestrcpy(name_local, p->name, 16);
                release(&p->lock);
                printf("%d,%s,%s\n", pid_local, state_local, name_local);
            }
        }

        if (unfound) {
            return 1;
        }
        return 0;
    }

        int pid_local;
        char state_local[16];
        char name_local[16];

    for(p = proc; p < &proc[NPROC]; p++) {

        acquire(&p->lock);
        if (p->pid != pid) {
            release(&p->lock);
            continue;
        }
        unfound = 0;
        pid_local = p->pid;
        safestrcpy(state_local, states[p->state], 16);
        safestrcpy(name_local, p->name, 16);
        release(&p->lock);
        break;
    }

    if (!unfound) {
        printf("%d,%s,%s\n", pid_local, state_local, name_local);
        return 0;
    }
```

```
    return 1;
}
```

0x22. define user program (ps)

- in `user/ps.c`

```
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

void
print_ps_header()
{
    printf("pid,state,name\n");
}

int
main(int argc, char *argv[])
{
    print_ps_header();
    if (argc == 1) {
        ps(0);
        exit(0);
    }
    for (int i = 1; i < argc; i++) {
        ps(atoi(argv[i]));
    }
    exit(0);
}
```

0x23. connect them

1가. wrapper

```
// sysproc.c
uint64
sys_ps(void)
{
  int pid;
  argint(0, &pid);
  return ps(pid);
}
```

1나. 왜 wrapper가 필요한가?

- register에 값을 넣어서 전달하기 때문에 user에서 `ps(atoi(argv[i]))`로 처리하더라도, 실제 구현은 해당 arg를 내부적으로는 register에 넘기고 다시 kernel에서 받아오는 구조이기 때문이다.
- syscall은 register 기반 ABI; Application Binary Interface를 통해 argument가 전달되며,
- kernel에서는 이를 argint, argaddr 등을 통해 C 타입으로 decode하고 검증해야 하므로 wrapper(sys_*)가 필요하다.

```
//in syscall.c
static uint64
argraw(int n)
{
  struct proc *p = myproc();
  switch (n) {
  case 0:
    return p->trapframe->a0;
  case 1:
    return p->trapframe->a1;
  case 2:
    return p->trapframe->a2;
  case 3:
    return p->trapframe->a3;
  case 4:
    return p->trapframe->a4;
  case 5:
    return p->trapframe->a5;
  }
  panic("argraw");
  return -1;
}

// Fetch the nth 32-bit system call argument.
void
argint(int n, int *ip)
{
  *ip = argraw(n);
}
```

2. system call 등록

```
// in syscall.c
extern uint64 sys_ps(void); //전역으로 정의된 sys_ps (in sysproc.c) 가져오기

static uint64 (*syscalls[])(void) = {
    [SYS_fork]    sys_fork,
    [SYS_exit]    sys_exit,
    [SYS_wait]    sys_wait,
    [SYS_pipe]    sys_pipe,
    [SYS_read]    sys_read,
```

```
    [SYS_kill]    sys_kill,
    [SYS_exec]    sys_exec,
    [SYS_fstat]   sys_fstat,
    [SYS_chdir]   sys_chdir,
    [SYS_dup]     sys_dup,
    [SYS_getpid]  sys_getpid,
    [SYS_sbrk]    sys_sbrk,
    [SYS_pause]   sys_pause,
    [SYS_uptime]  sys_uptime,
    [SYS_open]    sys_open,
    [SYS_write]   sys_write,
    [SYS_mknod]   sys_mknod,
    [SYS_unlink]  sys_unlink,
    [SYS_link]    sys_link,
    [SYS_mkdir]   sys_mkdir,
    [SYS_close]   sys_close,
    [SYS_ps]      sys_ps,
};
void
syscall(void)
{
  int num;
  struct proc *p = myproc();

  num = p->trapframe->a7;
  if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    p->trapframe->a0 = syscalls[num]();
  } else {
    printf("%d %s: unknown sys call %d\n",
            p->pid, p->name, num);
    p->trapframe->a0 = -1;
  }
}
```

```
// in syscall.h
// System call numbers
#define SYS_fork    1
#define SYS_exit    2
#define SYS_wait    3
#define SYS_pipe    4
#define SYS_read    5
#define SYS_kill    6
#define SYS_exec    7
#define SYS_fstat   8
#define SYS_chdir   9
#define SYS_dup    10
#define SYS_getpid 11
#define SYS_sbrk   12
```

```
#define SYS_pause  13
#define SYS_uptime 14
#define SYS_open   15
#define SYS_write  16
#define SYS_mknod  17
#define SYS_unlink 18
#define SYS_link   19
#define SYS_mkdir  20
#define SYS_close  21
#define SYS_ps     22
```

3. user program에서 system call은 어떻게 실행되는가

> map

```
user code
 → usys (li a7, SYS_ps)
 → ecall
 → hardware trap 발생
 → stvec 주소로 점프 (stvec = trap handler 주소)
 → uservec 실행
 → usertrap()
 → syscall()
 → sys_ps()
```

```
// in usys.pl
sub entry {
    my $prefix = "sys_";
    my $name = shift;
    if ($name eq "sbrk") {
    print ".global $prefix$name\n";
    print "$prefix$name:\n";
    } else {
    print ".global $name\n";
    print "$name:\n";
    }
    print " li a7, SYS_${name}\n";
    print " ecall\n";
    print " ret\n";
}

// ...
entry("ps")
```

위 코드는 사실상 아래와 같다.

```
ps:
  li a7, SYS_ps   // syscall 번호
  ecall           // kernel 진입
  ret             // 결과 반환
```

ecall을 하면 HW(cpu)에 trap이 발생한다.

- cpu가 stvec를 읽어 jump
- uservec이 설정되어 있을 것이므로 (prepare_return이 마지막에 항상 실행되고 그 후에 system call이 실행될 것이기 때문에)

> 그럼 uservec으로 이동해 `trampoline.S`의 다음 코드가 실행된다.

```
uservec:
        csrw sscratch, a0
        li a0, TRAPFRAME

        # save the user registers in TRAPFRAME
        sd ra, 40(a0)
        ...
        sd t6, 280(a0)

        # save the user 00 in p->trapframe->a0
        csrr t0, sscratch
        sd t0, 112(a0)

        # initialize kernel stack pointer, from p->trapframe->kernel_sp
        ld sp, 8(a0)

        # make tp hold the current hartid, from p->trapframe-
>kernel_hartid
        ld tp, 32(a0)

        # load the address of usertrap(), from p->trapframe->kernel_trap
        ld t0, 16(a0)

        # fetch the kernel page table address, from p->trapframe-
>kernel_satp.
        ld t1, 0(a0)

        # wait for any previous memory operations to complete, so that
        # they use the user page table.
        sfence.vma zero, zero

        # install the kernel page table.
        csrw satp, t1
        sfence.vma zero, zero
```

```
        # call usertrap()
        jalr t0
```

`usertrap()` in `trap.c`

```
uint64
usertrap(void)
{
  int which_dev = 0;

  if((r_sstatus() & SSTATUS_SPP) != 0)
    panic("usertrap: not from user mode");

  w_stvec((uint64)kernelvec);  //DOC: kernelvec

  struct proc *p = myproc();
  p->trapframe->epc = r_sepc();

  if(r_scause() == 8){
    // system call

    if(killed(p))
      kexit(-1);

    p->trapframe->epc += 4;

    intr_on();

    syscall();
  } else if((which_dev = devintr()) != 0){
    // ok
  } else if((r_scause() == 15 || r_scause() == 13) &&
            vmfault(p->pagetable, r_stval(), (r_scause() == 13)? 1 : 0) !=
0) {
    // page fault on lazily-allocated page
  } else {
    printf("usertrap(): unexpected scause 0x%lx pid=%d\n", r_scause(), p-
>pid);
    printf("            sepc=0x%lx stval=0x%lx\n", r_sepc(), r_stval());
    setkilled(p);
  }

  if(killed(p))
    kexit(-1);

  // give up the CPU if this is a timer interrupt.
  if(which_dev == 2)
    yield();
```

```
  prepare_return();

  // the user page table to switch to, for trampoline.S
  uint64 satp = MAKE_SATP(p->pagetable);

  // return to trampoline.S; satp value in a0.
  return satp;
}
```

> 이후에는 syscall() → syscalls[num]() → sys_ps() → #0x21의 흐름으로 결국 ps(pid)가 실행됨.

0x24. system call이 끝나면?

> `prepare_return()` in `trap.c`

```
void
prepare_return(void)
{
  struct proc *p = myproc();
  intr_off();

  uint64 trampoline_uservec = TRAMPOLINE + (uservec - trampoline);
  w_stvec(trampoline_uservec);

  p->trapframe->kernel_satp = r_satp();         // kernel page table
  p->trapframe->kernel_sp = p->kstack + PGSIZE; // process's kernel stack
  p->trapframe->kernel_trap = (uint64)usertrap;
  p->trapframe->kernel_hartid = r_tp();         // hartid for cpuid()

  unsigned long x = r_sstatus();
  x &= ~SSTATUS_SPP; // clear SPP to 0 for user mode
  x |= SSTATUS_SPIE; // enable interrupts in user mode
  w_sstatus(x);

  w_sepc(p->trapframe->epc);
}
```

0x30. PS

> trouble shooting 및 궁금증 해결

0x31. meta적 이해

> 지금 이 xv6 정보들은 어디에 위치되어 관리되는 것인가?

- C / asm → compiler → object file → linker → ELF binary (이 binary들이 결국 memory에 올라가는 것)
- 이 코드들은 모든 process address space에 존재(접근 불가)
  - 존재한다는 건, 모든 process별 page table에 kernel에 연결되어 있다는 것으로, 매핑 정보가 process's page table(VA->PA mapping에 존재한다는 것.
  - 물론, mode로 접근을 제어.

그럼 처음에는 어떻게 존재하는가?

1. build -> binary

1. bootloader:
   - ELF를 읽고
   - 각 segment를 physical memory에 load
2. CPU:
   - ELF entry point로 jump
   - kernel execution 시작

2. kernel memory 정의

1. kernel page table 생성 (kernel virtual address space 정의)
   - code
   - data
   - device memory
   - physical memory mapping
2. paging enable

3.init process (PID 1) 생성

1. kernel이 process 구조체 생성
2. 새 page table 생성
3. kernel mapping copy
4. user program (init code) load

0x32. design1 — buffer pointer를 정의해서 register 로 넘겨서 kernel이 거기에 쓰게 하자.

> system call - user program간 정보 전달 고민

user program에게 필요한 정보를 어떻게 전달할 것인가?

- 지금 user program에서 필요한 정보는 pid(int), state(string?), name(string?)인데
- system call 자체만으로 모든 정보를 전달할 수 없음

    - user program이 내부 정보에 직접 접근해서는 안됨
- 그럼 이정보들을 이때 바로 register로? 아니면 memory block에 넣고 주소만 register에?
    - string을 넘길 수는 없으므로
        - 8B(register 크기)를 넘는 경우, register수 제한까지 고려해야하는 복잡성.
    - buffer를 정의하고 해당 주소를 넘기자.
    - buffer는 stack or heap? 이건 추후 고민

> ∴ user program이 해야 할것: user process의 어디(buffer)에 해당 정보를 쓰도록 만들 것인지 그 위치(buffer pointer)를 인자로 넘겨야 함.

kernel에서 해당 정보를 어떻게 쓰지?

- kernel: syscall이 호출될 때 내부 함수에서 해당 정보를 쓸 수 있게 해야함.

`copyout`에는 해당 system call을 호출한 user process의 page table을 넣어야 하는데 어떻게 전달하지?

맥락: 왜 page table을 넣어야 할까?

```
- 넘겨준 memory block은 해당 프로세스의 [[VA; Virtual Address]] 기준이므로
```

- inner mechanism of copyout
    - walk(pagetable, va)
        - → PA; Physical Address 얻음
        - → memmove(pa, src, len)

가설1. memory block 이용?

> 그럼 user program에서 전달하는 memory block에 page_table pointer도 포함하면 되지 않나?

- 절대 안됨!
- kernel Only data를 user가 알게 해서는 안된다. 만약 그렇게 해버리면 arbitrary access를 허용하게 됨.

가설2. myproc을 그냥 사용해도 되는가?

```c
// Return the current struct proc *, or zero if none.
struct proc*
myproc(void)
{
    push_off();
    struct cpu *c = mycpu();
    struct proc *p = c->proc;
    pop_off();
```

```
    return p;
}
```

- 이걸 사용해도 되나?
  - 이걸 사용한다는 전제는 address space에 존재하는 kernel code를 사용한다는 것
- 지금 그럼 system call을 썼을 때 context switch를 막아야 하지 않나?
  - system call호출은 원자적으로 일어나야 하는가 그럼?

결론

- 원자적으로 일어날 필요 없음.
  1. critical section이 아님. 지금 실행되는 코드는
  2. context switch가 일어나더라도 current proc의 kernel context와 kernel stack이 저장되고, 다시 그 proc로 돌아오면 같은 syscall execution이 이어진다.
- 따라서 copyout이 잘못된 proc의 pagetable을 쓰는 문제는 생기지 않는다.

0x33. 방법1의 오류

> console에 직접 출력하면 되는데..

기존 IMPL

> memory block을 정의해서 pointer를 전달하게 하자.

```
#define NAME_LEN 16
#define STATE_LEN 16

struct psInfo {
    int pid;
    char name[NAME_LEN];
    char state[STATE_LEN];
};
```

```
int
ps(int pid)
{
    static char *states[] = {
        [UNUSED] "unused",
        [USED] "used",
        [SLEEPING] "sleep",
        [RUNNABLE] "runble",
        [RUNNING] "run",
        [ZOMBIE] "zombie"
    };

    struct proc *p;
```

```
    int unfound = 1;

    uint64 bp; // for getting buffer pointer
    argaddr(0, &bp); // 체크할 필요없음. copyout/in 때 수행함.

    struct psInfo kps;

    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if (p->pid != pid) {
            release(&p->lock);
            continue;
        }
        unfound = 0;
        kps.pid = pid;
        safestrcpy(kps.state, states[p->state], STATE_LEN);
        safestrcpy(kps.name, p->name, 16);
        release(&p->lock);
        break;
    }

    if (unfound || copyout(myproc()->pagetable, bp, (char*)&kps,
sizeof(kps)) == -1) {
        return 1;
    }

    return 0;
}
```

but, 문제점

- 현재 방법으로는 시그니처를 유지하면서 argument를 전달할 수가 없음
- console에 kernel이 직접 쓰게 하자

0x40. RESULT

basic direct test case

- case A — pid == 0, active process 전체 출력
- case B — 존재하는 pid 조회

- case C — 존재하지 않는 pid 조회

